	**ESCUELA DE INGENIERÍA DE SISTEMAS** **GUÍA DE LABORATORIO N° 03** **IS - 388**	**Código:** IS-388-LAB03 **Semana: 03** **Duración:** 2 horas
	Título: Simulación de Sprint y flujo de integración colaborativo en Jira y GitHub	Docente: Ing. Pelayo Quispe Bautista

1. INTRODUCCIÓN

En los laboratorios anteriores se construyó clases *Curso*, *Seccion* y *Calificacion*, aplicando código limpio y el ciclo rojo-verde-refactor. Ese trabajo técnico, sin embargo, se hizo como ejercicio individual y sin un plan compartido. En un proyecto real, el código se construye **en equipo**, en ciclos cortos y con acuerdos explícitos sobre qué se va a hacer, cuánto cuesta y cuándo algo está realmente terminado.

Scrum organiza ese trabajo en *Sprints*: periodos fijos, de una a cuatro semanas, al final de los cuales el equipo entrega un incremento de producto utilizable. Schwaber y Sutherland (2020) lo describen como un marco de trabajo **liviano** que no dice cómo programar, sino cómo planificar, inspeccionar y adaptar el trabajo mediante roles, eventos y artefactos bien definidos. Jira y GitHub Projects son las plataformas que usaremos para hacer visible ese plan: el *backlog*, las estimaciones y el avance del Sprint.

Esta guía simula el **Sprint 1 del Proyecto**. El equipo configurará un espacio Scrum en Jira, redactará historias de usuario con criterios de aceptación, las estimará con *Planning Poker*, acordará su **Definition of Done** (DoD) técnica y cerrará el ciclo con el **flujo de integración colaborativo** en GitHub: rama por incidencia, *Pull Request* y revisión por pares antes de integrar a main. Como señala el sílabo, el objetivo no es solo usar una herramienta, sino practicar el rol de *Developer Senior*: refinar, estimar con criterio técnico y cuidar la calidad de lo que el equipo integra.

2. COMPETENCIAS Y LOGROS DE APRENDIZAJE

2.1 Competencia General

Gestionar y planificar un Sprint aplicando Scrum en una plataforma ágil (Jira o GitHub Projects), redactando y estimando técnicamente el *Product Backlog*, estableciendo una *Definition of Done* verificable y aplicando un flujo de integración colaborativo basado en ramas, *Pull Requests* y revisión por pares.

2.2 Logros Específicos

- Explicar los roles, eventos y artefactos de Scrum y relacionarlos con las funcionalidades de Jira (espacio, backlog, sprint, tablero e informes).
- Redactar historias de usuario en el formato *Como quiero para* con criterios de aceptación en formato **Dado-Cuand-Entonces**.
- Estimar historias en *story points* mediante *Planning Poker*, justificando las diferencias de estimación con argumentos técnicos.

- Diferenciar la *Definition of Ready* (DoR) de la *Definition of Done* (DoD) y redactar una DoD técnica verificable para el equipo.
- Planificar e iniciar el Sprint 1 en Jira con un **objetivo de Sprint**, un compromiso acorde a la capacidad del equipo y un tablero configurado.

3. FUNDAMENTO TEÓRICO

3.1 Scrum en la construcción de software

Scrum se apoya en el empirismo: las decisiones se toman con base en lo que se observa, y por eso todo el trabajo se hace **transparente** para poder **inspeccionarlo** y **adaptarlo** en cada evento.

> **Scrum** es un marco de trabajo liviano que ayuda a personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos. Un *Scrum Team* está formado por un *Product Owner*, un *Scrum Master* y los *Developers*, y trabaja en *Sprints* de duración fija que producen un *Incremento* que cumple la *Definition of Done*.
>
> *Schwaber, K. y Sutherland, J. (2020)*

Elemento	Nombre	Propósito y equivalente en Jira
Rol	Product Owner	Maximiza el valor del producto: ordena el *Product Backlog* y decide qué se hace primero. En Jira prioriza arrastrando incidencias en el backlog.
Rol	Scrum Master	Asegura que Scrum se entienda y se aplique; elimina impedimentos. Suele administrar el espacio y el tablero.
Rol	Developers	Construyen el Incremento. Estiman, se autoasignan tareas y mueven las incidencias por el tablero.
Evento	Sprint Planning	Define por qué (objetivo), qué (historias) y cómo (tareas) del Sprint. En Jira: *Crear sprint -> Iniciar sprint*.
Evento	Daily Scrum	Reunión de 15 minutos para inspeccionar el avance hacia el objetivo. Se hace frente al tablero del Sprint.
Evento	Sprint Review	El equipo muestra el Incremento a los interesados.
Evento	Retrospectiva	Inspecciona cómo trabajó el equipo y acuerda mejoras. Jira permite completar el sprint y revisar sus informes.
Artefacto	Product Backlog	Lista ordenada de todo lo que se necesita en el producto. Compromiso asociado: Objetivo del Producto.
Artefacto	Sprint Backlog	Historias seleccionadas más el plan para entregarlas. Compromiso asociado: Objetivo del Sprint.

Elemento	Nombre	Propósito y equivalente en Jira
Artefacto	Incremento	Trabajo terminado y utilizable. Compromiso asociado: Definition of Done.

3.2 Jerarquía de incidencias en Jira

En Jira toda unidad de trabajo es una **incidencia** (*issue* o *work item*) con una clave única formada por la clave del espacio y un número, por ejemplo MAT-5. Esa clave es la que después se usará en ramas, *commits* y *Pull Requests* para conectar el código con el plan.

Tipo	Qué representa	Ejemplo en el Sistema de Matrícula
Épica	Funcionalidad grande que agrupa varias historias; suele abarcar más de un Sprint.	MAT-2 · Matrícula en secciones
Historia	Funcionalidad descrita desde la perspectiva de la persona usuaria; cabe en un Sprint.	Como estudiante, quiero matricularme en una sección con vacantes…
Tarea	Trabajo técnico necesario que no es una funcionalidad visible para el usuario.	Configurar el repositorio y la ejecución de pruebas.
Subtarea	Paso concreto para completar una historia o tarea; se estima en horas o no se estima.	Escribir las pruebas unitarias de matricular_estudiante().
Error (Bug)	Comportamiento incorrecto que debe corregirse.	La sección acepta matrículas con capacidad 0.

3.3 Historias de usuario y criterios de aceptación

Una historia de usuario no es una especificación completa, sino una promesa de conversación: describe **quién** necesita algo, **qué** necesita y **para qué**. Los criterios de aceptación convierten esa conversación en condiciones comprobables.

> **CONCEPTO: Historia de usuario:** descripción breve de una funcionalidad escrita desde la perspectiva de quien la desea, con la forma *Como [rol], quiero [acción] para [beneficio]*. Se acompaña de **criterios de aceptación** que indican cuándo la historia se considera cumplida.
>
> *Cohn, M. (2004)*

Los criterios de aceptación se redactan en formato **Dado - Cuando - Entonces** (*Given – When -Then*), que se corresponde directamente con el patrón AAA de las pruebas unitarias: *Dado* = Organizar, *Cuando* = Actuar, *Entonces* = Verificar.

Historia MAT-5: Como estudiante, quiero matricularme en una sección con vacantes para asegurar mi lugar en el curso.

Criterio 1 (camino feliz)
Dado una sección IS-282-A con capacidad 30 y 0 matriculados
Cuando el estudiante se matricula
Entonces matriculados es 1 y vacantes_disponibles() es 29

Criterio 2 (caso borde)
Dado una sección con capacidad 1 y 1 matriculado
Cuando otro estudiante intenta matricularse
Entonces matriculados sigue en 1 y se informa "No hay vacantes en IS-282-A"

Una buena historia cumple los criterios **INVEST** propuestos por Wake (2003):

Letra	Criterio	Pregunta de verificación
I	Independiente	¿Se puede desarrollar sin esperar a otra historia?
N	Negociable	¿Deja espacio para conversar el detalle con el Product Owner?
V	Valiosa	¿Entrega valor visible a alguna persona usuaria?
E	Estimable	¿El equipo entiende lo suficiente como para estimarla?
S	Pequeña (*Small*)	¿Cabe holgadamente en un Sprint?
T	Verificable (*Testable*)	¿Tiene criterios de aceptación que se puedan comprobar con pruebas?

3.4 Definition of Ready (DoR) y Definition of Done (DoD)

Ambas definiciones son *acuerdos del equipo*, pero se aplican en momentos distintos: la *DoR es un filtro* ***de entrada*** *al Sprint* y la *DoD es un filtro* ***de salida***. La Guía de Scrum solo exige la DoD; la DoR es una práctica complementaria muy usada para evitar que entren al Sprint historias ambiguas.

> **Definition of Done (DoD):** descripción formal del estado del Incremento cuando cumple las medidas de calidad requeridas para el producto. En el momento en que un elemento del Product Backlog cumple la DoD, nace un Incremento. Si no la cumple, no puede presentarse en la Sprint Review y regresa al Product Backlog.
>
> *Schwaber, K. y Sutherland, J. (2020)*

Aspecto	Definition of Ready (DoR)	Definition of Done (DoD)
Pregunta	¿La historia está lista para empezar?	¿La historia está realmente terminada?
Momento	Refinamiento, antes del Sprint Planning.	Durante el Sprint, antes de mover a *Listo*.
Alcance	Cada historia individual.	Todo el trabajo del equipo (es común a todas las historias).
Ejemplo de ítem	Tiene criterios de aceptación en Dado -Cuando - Entonces.	Las pruebas unitarias pasan y el PR fue aprobado por un compañero.
Si no se cumple	La historia no entra al Sprint; vuelve a refinamiento.	La historia no cuenta para la velocidad; sigue abierta.

3.5 Estimación técnica: story points y Planning Poker

Los *story points* son una unidad **relativa**: no miden horas, sino el tamaño de una historia en comparación con otras, *combinando esfuerzo, complejidad e incertidumbre*. Cohn (2005) recomienda usar una escala con saltos crecientes "*la serie de Fibonacci modificada"* porque cuanto más grande es una historia, menos precisa puede ser su estimación.

Puntos	Interpretación orientativa para el equipo
1	Trivial: un cambio pequeño y conocido (por ejemplo, un método consultor con su prueba).
2 – 3	Pequeña: una clase o método con validación, sin dependencias externas.
5	Mediana: varias clases o reglas de negocio con casos borde; algo de incertidumbre.
8	Grande: límite superior aceptable para un Sprint; conviene revisar si se puede dividir.
13 o más	Demasiado grande o incierta: se debe dividir antes de entrar al Sprint.
?	No hay información suficiente para estimar: la historia necesita refinamiento.

En **Planning Poker** cada Developer elige en secreto una carta, todas se muestran a la vez y, si hay diferencias grandes, quienes dieron el valor más alto y el más bajo explican sus razones. Se repite la votación hasta converger. El valor del ejercicio no está en el número final, sino en la conversación: las diferencias revelan supuestos técnicos que el resto del equipo no había considerado.

> **Velocidad:** cantidad de story points de las historias que cumplieron la DoD al final de un Sprint. En el primer Sprint no existe velocidad histórica, por lo que el compromiso se basa en la **capacidad** del equipo (horas disponibles) y en el juicio del equipo; a partir del Sprint 2 se usa la velocidad observada.
>
> *Cohn, M. (2005)*

3.6 Flujo de integración colaborativo con Pull Requests

El flujo que usará el equipo es una variante de *GitHub Flow*: main siempre contiene código que funciona, y todo cambio entra a main únicamente a través de un *Pull Request* revisado. Cada rama nace de una incidencia de Jira y lleva su clave en el nombre, lo que permite que Jira muestre automáticamente las ramas, *commits* y PR asociados.

Paso	Acción	Convención del curso
1	Crear rama desde main actualizada	feature/MAT-5-matricular-estudiante (tipo/clave-descripción corta)
2	Hacer *commits* pequeños con TDD	Mensaje que empieza con la clave: MAT-5 Agrega validación de vacantes
3	Publicar la rama y abrir el PR	Título: MAT-5: Matricular estudiante en sección con vacantes; completar la plantilla
4	Revisión por pares (*code review*)	Al menos 1 aprobación de un compañero distinto al autor
5	Integrar (*merge*) y cerrar	Solo si cumple la DoD; se elimina la rama y la incidencia pasa a *Listo*

> **Pull Request (PR):** solicitud para integrar los cambios de una rama en otra (normalmente main). Permite discutir el cambio, ver las diferencias línea por línea, ejecutar comprobaciones automáticas y registrar la aprobación o el pedido de cambios de quienes lo revisan antes de integrarlo.
>
> *GitHub Docs (2026)*

3.7 Buenas prácticas al gestionar un Sprint

- Definir primero el **objetivo del Sprint** y después elegir las historias que lo cumplen, no al revés.
- No comprometer más del 70 a 80 % de la capacidad calculada en el primer Sprint: siempre aparecen imprevistos (configuración, dudas, dependencias).
- Dividir toda historia de 13 puntos o más antes de planificar; una historia que no se termina no suma velocidad.
- Mantener el tablero actualizado **antes** de la Daily Scrum: una tarjeta que no se mueve en dos días es una señal de impedimento.
- Mantener los PR pequeños (idealmente menos de 400 líneas modificadas) para que la revisión sea rápida y útil.
- Revisar el código, no a la persona: los comentarios de revisión deben ser concretos, justificados y proponer una alternativa.

4. MATERIALES Y REQUISITOS

- Laptop o computador del laboratorio con conexión a internet y un navegador actualizado.
- Cuenta de correo (se recomienda Gmail o el correo institucional) para registrarse en Atlassian.
- Cuenta de GitHub.
- Git instalado y configurado
- Equipos de 4 integrantes con roles asignados: 1 Product Owner, 1 Scrum Master y Developers.

5. DESARROLLO DEL LABORATORIO

Durante todo el laboratorio se trabaja sobre el **Sistema de Matrícula**.

A1 CREAR LA CUENTA ATLASSIAN Y EL SITIO DE JIRA

Solo **el Scrum Master** de cada equipo realiza esta actividad; el resto del equipo será invitado en la Actividad A2. El plan gratuito de Jira admite hasta 10 usuarios, suficiente para el equipo.

Paso 1. Ingresen a www.atlassian.com/software/jira y hagan clic en **Get it free** (Obténlo gratis), o vayan directamente a https://id.atlassian.com/signup.

Figura 1.
Página principal de Jira con el botón «Get it free».

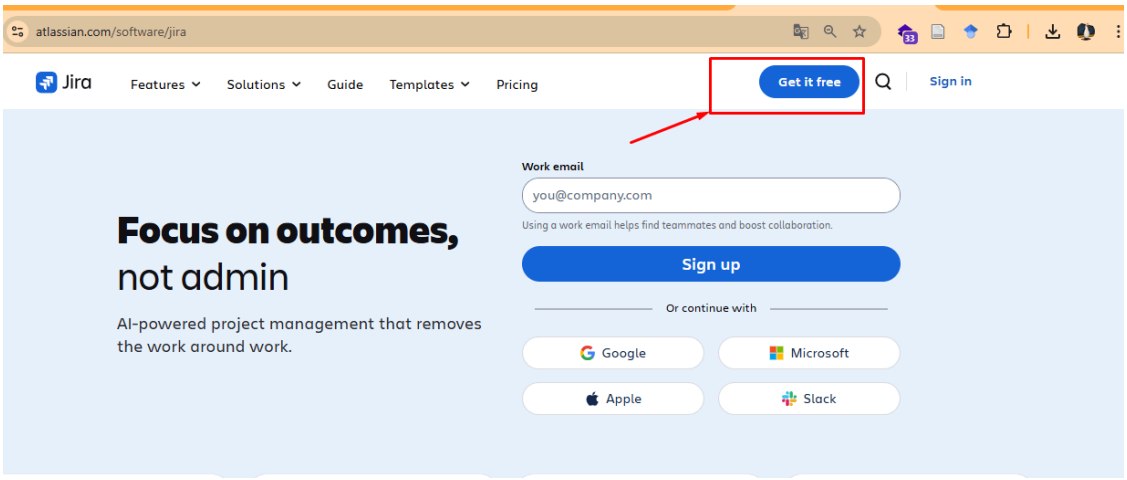
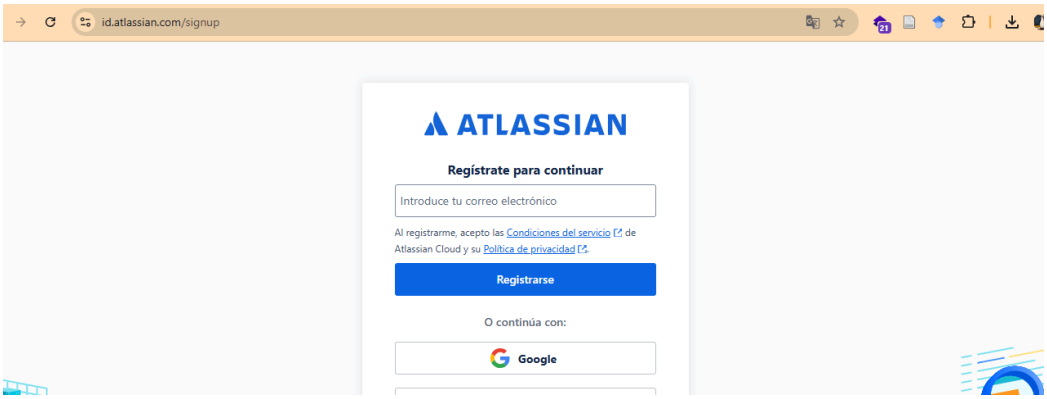
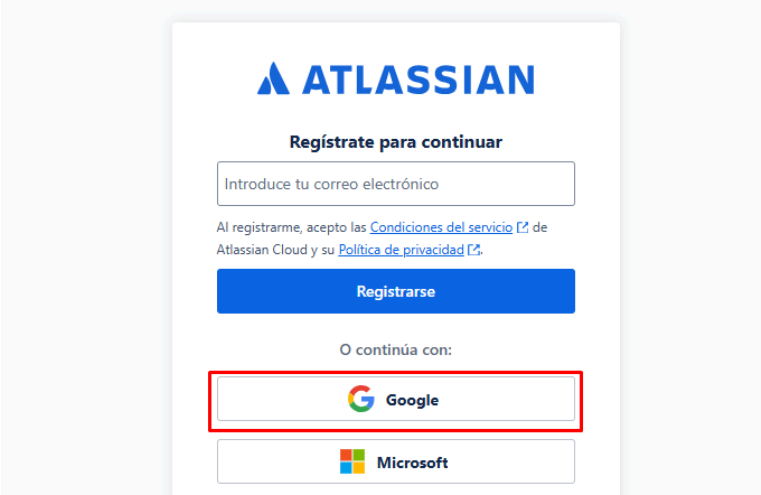


Figura 2.
Formulario de registro directo en id.atlassian.com/signup.



Paso 2. Regístrense con **Continuar con Google** (opción más rápida) o con correo y contraseña. Si usan correo, verifiquen la cuenta desde el enlace que Atlassian envía a su bandeja de entrada.

Figura 3.
Registro con la opción «Continuar con Google».



Paso 3. Al ingresar verán la página de inicio de Atlassian. Abran el **selector de aplicaciones** (ícono de cuadrícula, arriba a la izquierda) y elijan **Jira**.

Figura 4.
Página de inicio de Atlassian después de iniciar sesión.

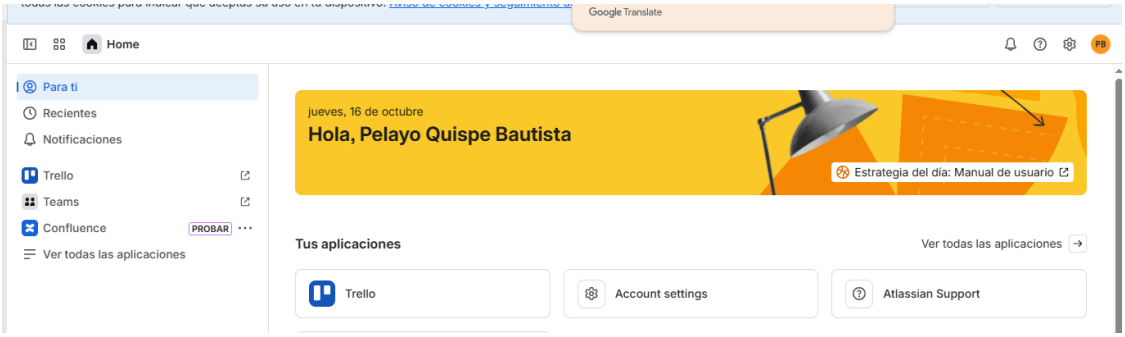
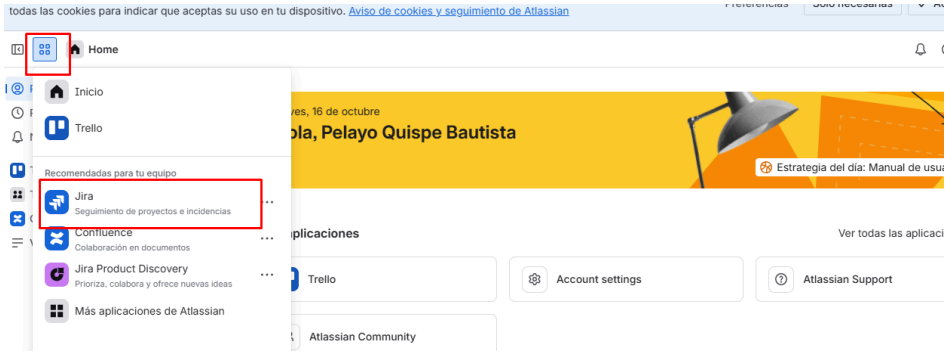


Figura 5.
Selector de aplicaciones de Atlassian con la opción Jira resaltada.



Paso 4. En la página de Jira hagan clic en **Probar ahora** y, a continuación, escriban el nombre de su **espacio de trabajo** (sitio). Usen el formato matricula-unsch-eqXX, donde XX es el número de su equipo.

Figura 6.
Página de Jira «Desarrollo de software para equipos ágiles» con el botón «Probar ahora».



Figura 7.
Creación del espacio de trabajo: el nombre define la URL [nombre].atlassian.net.



> **OJO:** El nombre del sitio forma la URL [nombre].atlassian.net y **no se puede cambiar después**. Usen solo minúsculas, números y guiones. Si aparece una X roja, el nombre ya está tomado: prueben una variación.

Paso 5. Confirmen que están en el **plan Free**, revisen la dirección del espacio de trabajo y hagan clic en **Empezar**. Esperen a que Jira termine de preparar el sitio y pulsen **Ir a Jira**.

Figura 8.
Confirmación del plan Free y de la dirección del espacio de trabajo.

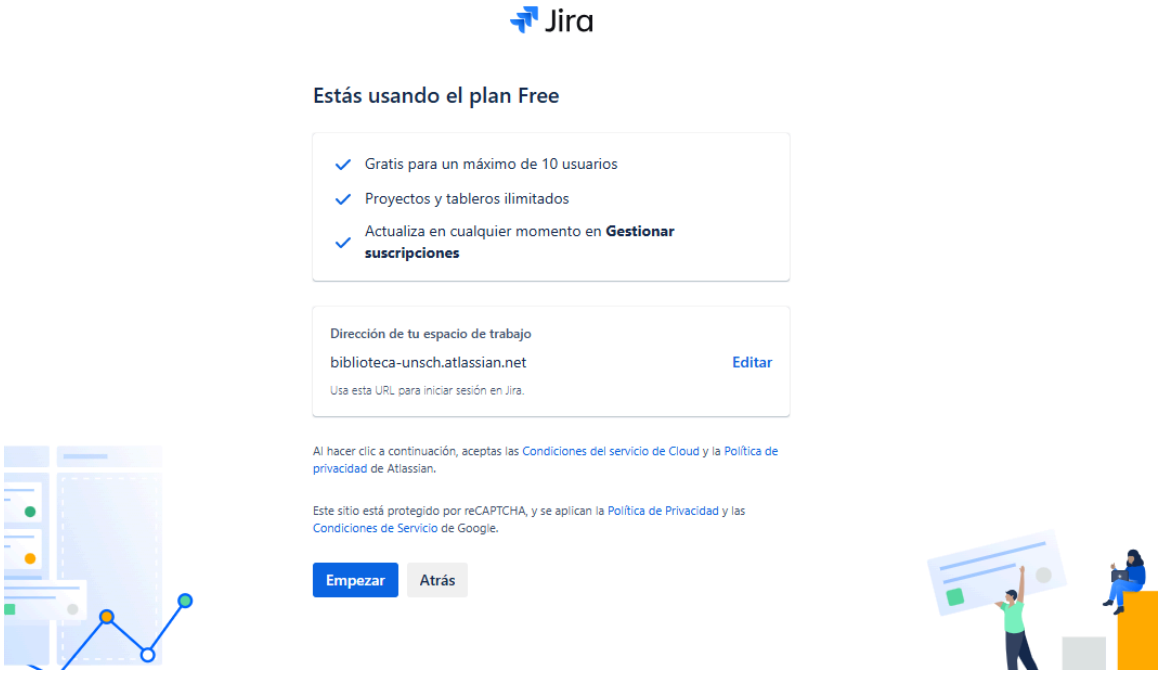


Figura 9.
Jira preparando el sitio.



Figura 10.
Sitio listo: botón «Ir a Jira».



A2 CREAR EL ESPACIO SCRUM Y CONFIGURAR EL TABLERO

Paso 1. Cuando Jira pida una plantilla para el primer espacio, seleccionen **Scrum** (no Kanban): Scrum habilita el backlog, los sprints y los informes de avance que se usarán en esta guía. Hagan clic en **Siguiente**.

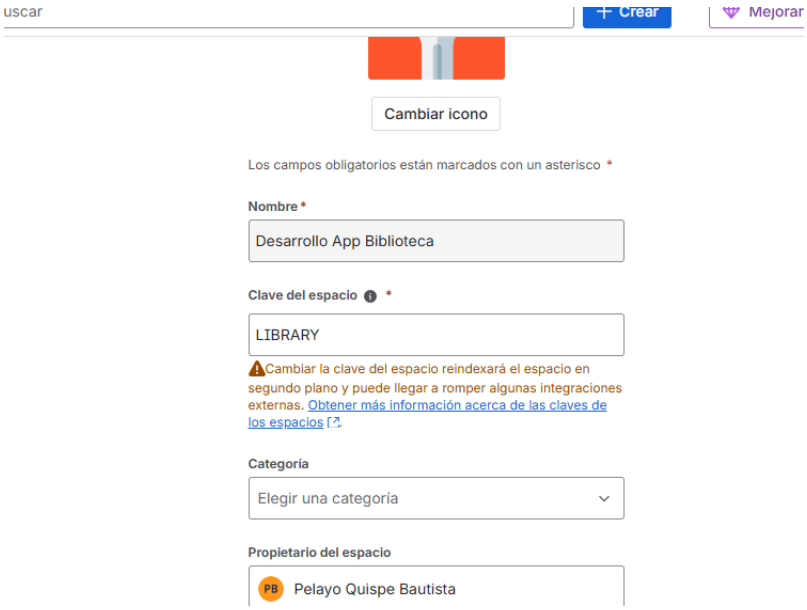
Figura 11.
Selección de la plantilla Scrum para el primer espacio.



Paso 2. Completen los datos del espacio. La **clave** es muy importante: será el prefijo de todas las incidencias y la usarán en ramas y *commits*.

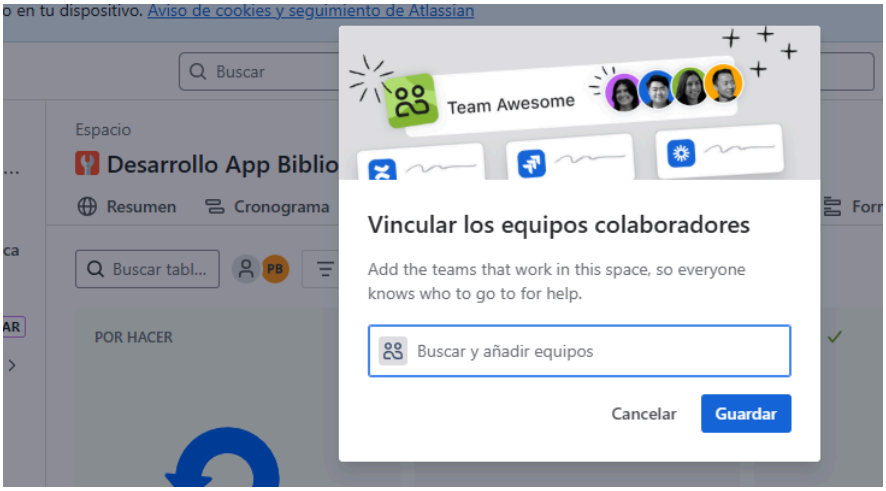
Campo	Valor para el equipo	Observación
Nombre	Sistema de Matrícula UNSCH – Equipo XX	Visible para todos los miembros.
Clave del espacio	MAT	Las incidencias serán MAT-1, MAT-2… Cambiarla después reindexa el espacio y puede romper integraciones.
Propietario	Scrum Master del equipo	Luego se agregan los demás integrantes.

Figura 12.
Datos del espacio: nombre, clave y propietario.



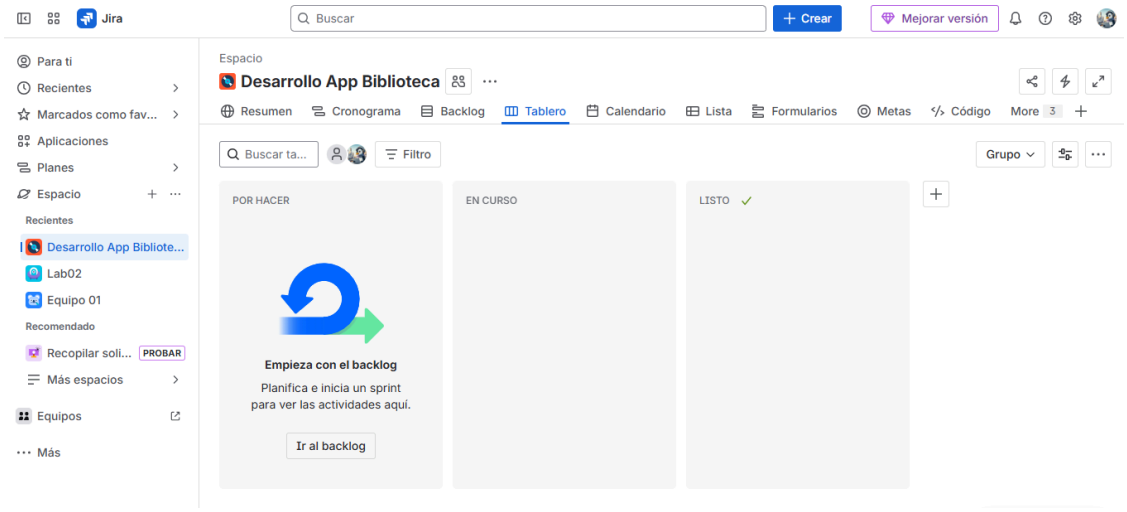
Paso 3. Si aparece el diálogo **Vincular los equipos colaboradores**, pueden cancelarlo. Luego inviten a los integrantes y al docente desde **Configuración del espacio -> Acceso** (o con el botón **Añadir personas**) usando sus correos.

Figura 13.
Diálogo opcional para vincular equipos colaboradores.



Paso 4. Abran la pestaña **Tablero**. Por defecto tiene tres columnas: POR HACER, EN CURSO y LISTO. Agreguen una columna **EN REVISIÓN** entre EN CURSO y LISTO con el botón **+**: allí estarán las historias cuyo PR espera aprobación.

Figura 14.
Tablero Scrum recién creado, con las columnas por defecto y el botón + para agregar columnas.



Columna	Significado para el equipo	Regla de paso
POR HACER	Historia comprometida en el Sprint, aún sin iniciar.	Un Developer se la asigna al empezar.
EN CURSO	Se está desarrollando en su rama.	Pasa cuando se abre el PR.

Columna	Significado para el equipo	Regla de paso
EN REVISIÓN	PR abierto esperando revisión o cambios.	Pasa cuando el PR se integra a main.
LISTO	Cumple toda la DoD.	Solo el equipo, verificando la DoD, la mueve aquí.

A3 CONSTRUIR EL PRODUCT BACKLOG

El Product Owner dirige esta actividad con todo el equipo. En la pestaña **Backlog** creen primero las épicas y después las historias, asociando cada historia a su épica con el campo **Principal** (*Parent*).

Paso 1. Creen las tres épicas del sistema:

Clave	Épica	Descripción
MAT-1	Catálogo de cursos	Registro y consulta de cursos del plan de estudios.
MAT-2	Matrícula en secciones	Apertura de secciones por semestre y matrícula con control de vacantes.
MAT-3	Registro de calificaciones	Registro, corrección y consulta de notas de 0 a 20.

Paso 2. Creen las siguientes historias y la tarea técnica. Escriban los criterios de aceptación en el campo **Descripción** con el formato **Dado – Cuando - Entonces** de la sección 3.3.

Clave	Tipo	Historia de usuario	Épica
MAT-4	Tarea	Configurar el repositorio, la plantilla de PR y la ejecución de pruebas con unittest y coverage.	—
MAT-5	Historia	Como coordinador académico , quiero registrar un curso con su código, créditos y horas, para mantener el catálogo actualizado.	MAT-1
MAT-6	Historia	Como coordinador académico, quiero consultar el total de horas de un curso, para asignar la carga docente.	MAT-1
MAT-7	Historia	Como coordinador académico, quiero abrir una sección de un curso en un semestre con una capacidad máxima, para ofertarla a los estudiantes.	MAT-2
MAT-8	Historia	Como estudiante, quiero matricularme en una sección con vacantes, para asegurar mi lugar en el curso.	MAT-2
MAT-9	Historia	Como estudiante, quiero recibir un mensaje claro cuando la sección no tiene vacantes, para elegir otra sección.	MAT-2

Clave	Tipo	Historia de usuario	Épica
MAT-10	Historia	Como docente, quiero registrar la nota de un estudiante entre 0 y 20, para dejar constancia de su evaluación.	MAT-3
MAT-11	Historia	Como estudiante, quiero saber si aprobé un curso (nota ≥ 11), para planificar mi siguiente matrícula.	MAT-3
MAT-12	Historia	Como estudiante, quiero ver el reporte de todos mis cursos matriculados y sus notas en el semestre, para conocer mi situación académica.	MAT-2 / MAT-3

Paso 3. Ordenen el backlog arrastrando las incidencias: el Product Owner coloca arriba lo que más valor aporta y lo que otras historias necesitan primero (por ejemplo, no se puede matricular sin haber abierto una sección).

> **NOTA:** Noten que MAT-12 depende de las épicas MAT-2 y MAT-3 a la vez y describe una consulta amplia. Es candidata a no cumplir la **I** (independiente) ni la **S** (pequeña) de INVEST. La retomaremos en la Actividad A4.

A4 REFINAMIENTO Y ESTIMACIÓN CON PLANNING POKER

Paso 1 (DoR). Antes de estimar, revisen cada historia con la siguiente *Definition of Ready*. Una historia que no cumple algún punto vuelve al Product Owner para completarse.

#	Criterio de la Definition of Ready del equipo
1	Está redactada en el formato *Como - quiero - para* y asociada a una épica.
2	Tiene al menos dos criterios de aceptación en Dado - Cuando - Entonces (camino feliz y un caso borde).
3	No depende de otra historia que no esté terminada o incluida en el mismo Sprint.
4	El equipo la entiende lo suficiente para estimarla y su estimación es de 8 puntos o menos.

Paso 2 (Planning Poker). Usen cartas físicas, una aplicación de Planning Poker o simplemente los dedos, con la escala 1, 2, 3, 5, 8, 13, ?. Para cada historia: el PO la lee, el equipo pregunta, todos votan a la vez, discuten los extremos y vuelven a votar. Registren el valor final en el campo **Estimación de story points** de cada incidencia. El siguiente ejemplo muestra una ronda real para MAT-8:

Ronda	Dev 1	Dev 2	Dev 3	Dev 4	Discusión
1	3	5	5	13	Dev 4 votó 13 porque pensó que había que guardar la matrícula en base de datos. El PO aclara que en este Sprint solo se construye la lógica de dominio. Dev 1 votó 3 porque ya existe *Seccion*, pero Dev 2 recuerda que faltan las pruebas de casos borde.
2	5	5	5	5	Consenso: 5 puntos.

Paso 3 (Dividir). Al estimar MAT-12 es probable que el equipo obtenga 13 o «?». Divídanla en historias más pequeñas y verticales, por ejemplo: *(a)* listar los cursos matriculados en el semestre y *(b)* agregar a ese listado la nota y su condición de aprobado. Creen las nuevas historias, estímenlas y cierren MAT-12 o conviértanla en épica.

Como referencia, la estimación final que obtuvo un equipo de prueba fue la siguiente (sus valores pueden variar; lo importante es que puedan justificarlos):

Clave	Resumen	Puntos	Justificación técnica breve
MAT-4	Configurar repositorio y pruebas	2	Tarea conocida, pero involucra a todo el equipo.
MAT-5	Registrar curso	2	La clase *Curso* ya existe; falta validar datos de entrada.
MAT-6	Total de horas del curso	1	Método consultor ya probado en laboratorios previos.
MAT-7	Abrir sección con capacidad	3	Nueva validación: capacidad mayor que 0.
MAT-8	Matricular con vacantes	5	Reglas y casos borde; ver ronda de ejemplo.
MAT-9	Mensaje sin vacantes	2	Depende de MAT-8; cambia el manejo del error.
MAT-10	Registrar nota 0–20	3	Validación de rango con límites.
MAT-11	Condición de aprobado	2	Consultor con límite en 11.

A5 DEFINITION OF DONE Y PLANIFICACIÓN DEL SPRINT 1

Paso 1 (DoD). Acuerden en equipo la *Definition of Done* técnica. Debe ser verificable: cada ítem se responde con «sí» o «no», sin interpretaciones.:

#	Criterio de la Definition of Done del equipo	Cómo se verifica
1	El código está en una rama con la clave de la incidencia y se integró a main mediante un PR.	Panel *Desarrollo* de la incidencia en Jira.
2	Cada criterio de aceptación tiene al menos una prueba unitaria, y todas pasan.	python -m unittest en verde.
3	La cobertura del código de dominio nuevo es de al menos 80 %.	Reporte de coverage report.
4	El código respeta PEP 8 y no presenta *code smells* evidentes (nombres claros, métodos cortos, sin duplicación).	Linter sin errores y revisión del PR.
5	El PR fue aprobado por al menos un compañero distinto al autor y todos sus comentarios están resueltos.	Estado *Approved* en GitHub.
6	Después del *merge*, main sigue compilando y con todas las pruebas en verde.	Ejecución de pruebas sobre main.
7	Las clases y métodos públicos nuevos tienen *docstring* y el README se actualizó si cambió el uso.	Revisión del PR.

Publiquen la DoD donde todo el equipo la vea: en la descripción del espacio o en una página de resumen de Jira, y también en el archivo CONTRIBUTING.md del repositorio.

Paso 2 (Capacidad). Calculen la capacidad del equipo para un Sprint de 2 semanas. Como no existe velocidad histórica, esta será la referencia para el compromiso:

Integrante	Horas efectivas por semana	Semanas	Horas disponibles
Dev 1 (Product Owner)	5	2	10
Dev 2 (Scrum Master)	5	2	10
Dev 3	6	2	12
Dev 4	6	2	12
Total			44 h → comprometer ~75 %: 33 h

Paso 3 (Sprint Planning). En la pestaña **Backlog**, hagan clic en **Crear sprint**. Luego:

1. Redacten el **objetivo del Sprint** en una sola frase orientada a valor. Ejemplo: *«Un estudiante puede matricularse en una sección con vacantes de un curso del catálogo, con la lógica de dominio cubierta por pruebas unitarias.»*

2. Arrastren al Sprint, en orden, las historias que cumplen ese objetivo: MAT-4, MAT-5, MAT-6, MAT-7, MAT-8 y MAT-9 (**15 puntos**). Las historias de calificaciones quedan para el Sprint 2.
3. Descompongan cada historia en **subtareas** de no más de 4 horas (por ejemplo: «Pruebas de casos borde de MAT-8», «Implementar validación», «Abrir y atender PR»). Si la suma de horas de las subtareas supera la capacidad del Paso 2, devuelvan la última historia al backlog.
4. Hagan clic en **Iniciar sprint**, escriban el objetivo en el campo **Objetivo del sprint**, fijen la duración en 2 semanas y confirmen.

OJO: Observen que el objetivo del Sprint se definió **antes** de elegir historias. ¿Por qué MAT-10 (3 puntos) no entra al Sprint aunque el equipo tenga capacidad para ella? Relacionen su respuesta con el concepto de objetivo del Sprint de la sección 3.1.

A6 FLUJO DE INTEGRACIÓN COLABORATIVO: RAMA, PULL REQUEST Y REVISIÓN

Paso 1 (Vincular Jira con GitHub). El Scrum Master instala la aplicación **GitHub for Jira** desde **Aplicaciones -> Explorar más aplicaciones** en Jira, y conecta la cuenta u organización de GitHub donde está el repositorio del equipo. A partir de ese momento, toda rama, *commit* o PR que contenga una clave como MAT-8 aparecerá en el panel **Desarrollo** de esa incidencia.

Paso 2 (Proteger `main`). En GitHub, en **Settings -> Branches** (o **Rules -> Rulesets**), creen una regla para main que exija un *Pull Request* con **al menos 1 aprobación** antes de integrar. Así nadie, ni siquiera el propietario, puede hacer push directo a main.

Paso 3 (Plantilla de PR, tarea MAT-4). Creen el archivo .github/pull_request_template.md con la DoD como lista de verificación. GitHub la cargará automáticamente en cada PR nuevo:

```
## Incidencia de Jira
MAT-XX: <resumen de la historia>

## ¿Qué cambia?
-

## ¿Cómo se probó?
- [ ] Se vieron fallar antes de implementar
- [ ] `python -m unittest` en verde
- [ ] Cobertura del código nuevo >= 80 % (`coverage report`)

## Definition of Done
- [ ] Cada criterio de aceptación tiene al menos una prueba
- [ ] PEP 8 sin errores y sin code smells evidentes
- [ ] Docstrings en clases y métodos públicos nuevos
- [ ] README actualizado (si aplica)
```

Paso 4 (Desarrollar la historia). Cada Developer toma una historia del tablero, se la asigna y la mueve a **EN CURSO**. Luego crea su rama y trabaja, haciendo *commits* pequeños que incluyan la clave:

```
# 1. Partir siempre de main actualizada
git checkout main
git pull origin main

# 2. Crear la rama con la clave de la incidencia
git checkout -b feature/MAT-8-matricular-estudiante

# 3. commit del código
git add seccion.py
git commit -m "MAT-8 Valida vacantes en matricular_estudiante"

# 4. Verificar antes de publicar
python -m unittest
coverage run -m unittest && coverage report

# 5. Publicar la rama
git push -u origin feature/MAT-8-matricular-estudiante
```

Paso 5 (Abrir el PR). En GitHub hagan clic en **Compare & pull request**, usen el título MAT-8: Matricular estudiante en sección con vacantes, completen la plantilla, asignen como revisor (*Reviewer*) a un compañero y creen el PR. En Jira, muevan la historia a **EN REVISIÓN** y verifiquen que el PR aparece en su panel **Desarrollo**.

Paso 6 (Revisión por pares). Quien revisa abre la pestaña **Files changed**, comenta sobre líneas concretas y, al terminar, elige **Approve** o **Request changes** en **Review changes**. Usen esta guía mínima de revisión:

Aspecto	Pregunta que debe hacerse quien revisa
Criterios de aceptación	¿Cada criterio de la historia tiene una prueba que lo verifique?
Pruebas	¿Las pruebas son atómicas, independientes y siguen el patrón AAA?
Diseño	¿El código respeta SOLID y no duplica lógica existente?
Legibilidad	¿Los nombres explican la intención sin necesidad de comentarios?
Alcance	¿El PR hace solo lo que pide la historia, sin cambios ajenos?

Paso 7 (Integrar). Cuando el PR tenga aprobación y todos los comentarios resueltos, el autor hace **Merge pull request** y elimina la rama. Verifiquen la DoD completa sobre main y recién entonces muevan la historia a **LISTO**.

> **NOTA:** Una historia en **EN REVISIÓN** todavía no suma puntos. Si al final del Sprint el PR no se integró, la historia no está terminada según la DoD y vuelve al backlog: no existen historias «casi listas» ni «90 % terminadas».

A7 SEGUIMIENTO DEL SPRINT: DAILY SCRUM E INFORMES

Paso 1. Realicen una *Daily Scrum* simulada de 15 minutos frente al tablero. Cada integrante responde en relación con el **objetivo del Sprint**: qué hizo para acercarse a él, qué hará a continuación y qué impedimento tiene. El Scrum Master registra los impedimentos como comentarios en la incidencia afectada.

Paso 2. Abran **Informes** en el espacio de Jira y revisen el **gráfico de evolución del sprint** (*Sprint burndown*). Identifiquen la línea ideal y la línea real, y expliquen qué indicaría una línea real plana durante varios días.

Paso 3. Al cerrar el Sprint, usen **Completar sprint**: Jira moverá las historias no terminadas al backlog o al siguiente Sprint, y el informe de **Velocidad** mostrará los puntos comprometidos frente a los completados. Esa velocidad será la referencia para planificar el Sprint 2.

6. ACTIVIDADES DE CONSOLIDACIÓN

6.1 Trabajo en Equipo

Cada equipo debe aplicar todo lo trabajado en la sesión al **caso de negocio de su Proyecto Final** (no al Sistema de Matrícula), dejando el Sprint 1 configurado y en ejecución para la revisión técnica de la semana 7:

Entregable	Requisito mínimo
Product Backlog	Al menos 3 épicas y 10 historias de usuario en formato *Como - quiero - para*, con al menos dos criterios de aceptación Dado - Cuando - Entonces cada una.
Estimación	Todas las historias estimadas con Planning Poker; registrar en una tabla al menos 2 historias con más de una ronda de votación y la discusión que las hizo converger.
DoR y DoD	Ambas definiciones redactadas, verificables y publicadas en Jira y en CONTRIBUTING.md.
Sprint 1	Sprint iniciado con objetivo, historias descompuestas en subtareas y compromiso justificado con el cálculo de capacidad.

Entregable	Requisito mínimo
Flujo de integración	main protegida, plantilla de PR y al menos un PR por integrante, cada uno revisado y aprobado por otro miembro, con al menos un comentario de revisión atendido.
Trazabilidad	Jira vinculado con GitHub: las ramas, *commits* y PR aparecen en el panel *Desarrollo* de sus incidencias.

Ejercicio adicional (GitHub Projects). Repliquen el backlog y el Sprint 1 en **GitHub Projects** usando un campo de iteración (*Iteration*) para el Sprint y un campo numérico *Estimate* para los puntos. Comparen en un cuadro de media página ambas herramientas: configuración, trazabilidad con el código e informes disponibles.

6.2 Preguntas de Reflexión

1. ¿Por qué los *story points* son una medida relativa y no horas? ¿Qué problema aparecería si el docente comparara la velocidad de dos equipos distintos?
2. En la ronda de Planning Poker de la Actividad A4, la diferencia entre 3 y 13 puntos se explicó por un supuesto técnico distinto. Den un ejemplo de su propio backlog en el que la discusión haya revelado un supuesto oculto.
3. ¿Qué diferencia hay entre la *Definition of Ready* y la *Definition of Done*? ¿Qué ocurriría con la velocidad del equipo si la DoD no exigiera pruebas unitarias?
4. ¿Por qué conviene proteger la rama main y exigir revisión por pares, incluso en un equipo pequeño donde todos se conocen? Relacionen su respuesta con la tarjeta que quedó en EN REVISIÓN en la Actividad A6.

7. REPORTE DE LABORATORIO

7.1 Estructura del Reporte

- Carátula: título de la práctica, curso, docente, integrantes del equipo con su rol Scrum y fecha.
- URL del sitio de Jira y del repositorio de GitHub, con acceso otorgado al docente.
- Capturas de pantalla del backlog estimado, del Sprint 1 iniciado (con su objetivo) y del tablero con las cuatro columnas.
- La DoR y la DoD del equipo, y la tabla de cálculo de capacidad.
- Capturas de al menos un PR con comentarios de revisión y su aprobación, y del panel *Desarrollo* de la incidencia vinculada en Jira.
- Entregables del trabajo en equipo de la sección 6.1 y respuestas a las preguntas de reflexión de la sección 6.2.
- Conclusiones breves (máximo un párrafo) sobre lo aprendido en la sesión.

8. RECURSOS Y REFERENCIAS

Documentación técnica:

- Atlassian. (2026). *Jira Software Cloud: documentación de soporte*. https://support.atlassian.com/jira-software-cloud/
- Atlassian. (2026). *Integrate Jira with GitHub (GitHub for Jira)*. https://support.atlassian.com/jira-cloud-administration/docs/integrate-with-github/
- GitHub. (2026). *About pull requests*. https://docs.github.com/es/pull-requests
- GitHub. (2026). *About Projects*. https://docs.github.com/es/issues/planning-and-tracking-with-projects
- Python Software Foundation. (2026). *unittest — Unit testing framework*. https://docs.python.org/3/library/unittest.html

Referencias bibliográficas:

- Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.
- Cohn, M. (2005). *Agile Estimating and Planning*. Prentice Hall.
- Forsgren, N., Humble, J. y Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- McConnell, S. (2004). *Code Complete* (2.ª ed.). Microsoft Press.
- Rubin, K. S. (2012). *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley.
- Schwaber, K. y Sutherland, J. (2020). *La Guía de Scrum: la guía definitiva de Scrum, las reglas del juego*. https://scrumguides.org
- Wake, B. (2003). *INVEST in Good Stories, and SMART Tasks*. XP123. https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/